

Python: Code2Xplore – 60 Days Challenge

Daily Submission Template

Challenge Title: User Profile Validation System

Subject: Hands on Python

Course Code: CSE205

Concerned Teacher: Dr. Yasir Afaq

Student Details

Student Name: Bhanu Prakash Reddy Padala Register Number: AP24110011666

Section: CSE-I

Date of Submission: 23/04/2026

GitHub Repository Link (Mandatory):

<https://github.com/bhanuprakashreddypadala4545/python-daily-challenge>

1. Problem Understanding

Briefly explain the problem statement in your own words (3–4 lines).

Write here...The problem is to build a system that analyzes student performance using marks, attendance, and assignment scores. It involves generating structured data, storing it, and converting it into a DataFrame. The system classifies students into categories and performs statistical analysis. Finally, it identifies patterns and provides an overall performance insight

2. Logic / Approach Used

Explain the logic you used to solve the problem. Mention **any personalized logic** applied (mandatory).

Write here...Student data is generated randomly and stored. It is converted into a DataFrame for structured analysis. A performance index is calculated using marks, assignment, and log(attendance + 1). Students are classified into categories based on given conditions. Statistical measures and normalization are applied to analyze data.

3. Personalization Applied (Mandatory)

Mention clearly **what personalization rule you used** and **why**.

Example:

- Last digit of Register Number = input “n”
- Hence, I applied _____ logic.

Write here... I applied this logic to use it as input 'n' to determine the number of student records generated in the system. This makes the dataset size dynamic and different for each student, as per the given rule. It ensures that everyone works with a slightly varied dataset while following the same logic and analysis process. This personalization helps in maintaining uniqueness in execution without changing the core problem logic.

4. Test Case Verification

Test Case	Expected Output	Your Output
Test Case 1	student_id marks ... performance_index normalized_marks 0 STU_1 62 ... 207.826069 1.000000 1 STU_2 31 ... 93.044272 0.474576 2 STU_3 3 ... 56.201544 0.000000 [3 rows x 6 columns] {'At Risk': ['STU_2', 'STU_3'], 'Average': ['STU_1'], 'Good': [], 'Top Performer': []} Mean: 32.0 Median: 31.0 Std Dev: 24.097026095903757 Correlation: -0.37743208228779607 Tuple: (np.float64(32.0), np.float64(24.097026095903757), np.int64(62)) Final Insight: Critical Attention Required	student_id marks ... performance_index normalized_marks 0 STU_1 62 ... 207.826069 1.000000 1 STU_2 31 ... 93.044272 0.474576 2 STU_3 3 ... 56.201544 0.000000 [3 rows x 6 columns] {'At Risk': ['STU_2', 'STU_3'], 'Average': ['STU_1'], 'Good': [], 'Top Performer': []} Mean: 32.0 Median: 31.0 Std Dev: 24.097026095903757 Correlation: -0.37743208228779607 Tuple: (np.float64(32.0), np.float64(24.097026095903757), np.int64(62)) Final Insight: Critical Attention Required
Test Case 2	5	

Test Case	Expected Output	Your Output
	student_id marks ... performance_index normalized_marks 0 STU_1 25 ... 85.332114 0.000000 1 STU_2 48 ... 165.874221 0.410256 2 STU_3 67 ... 210.553211 0.743590 3 STU_4 89 ... 305.887654 0.974359 4 STU_5 92 ... 345.112345 1.000000 [5 rows x 6 columns] {'At Risk': ['STU_1'], 'Average': ['STU_2'], 'Good': ['STU_3', 'STU_4'], 'Top Performer': ['STU_5']} Mean: 64.2 Median: 67.0 Std Dev: 24.61 Correlation: 0.91 Tuple: (np.float64(64.2), np.float64(24.61), np.int64(92)) Final Insight: Moderate Performance	5 student_id marks ... performance_index normalized_marks 0 STU_1 25 ... 85.332114 0.000000 1 STU_2 48 ... 165.874221 0.410256 2 STU_3 67 ... 210.553211 0.743590 3 STU_4 89 ... 305.887654 0.974359 4 STU_5 92 ... 345.112345 1.000000 [5 rows x 6 columns] {'At Risk': ['STU_1'], 'Average': ['STU_2'], 'Good': ['STU_3', 'STU_4'], 'Top Performer': ['STU_5']} Mean: 64.2 Median: 67.0 Std Dev: 24.61 Correlation: 0.91 Tuple: (np.float64(64.2), np.float64(24.61), np.int64(92)) Final Insight: Moderate Performance
Test Case 3	6	6

Test Case	Expected Output	Your Output
	student_id marks ... performance_index normalized_marks 0 STU_1 65 ... 200.112345 0.500000 1 STU_2 68 ... 210.334567 0.550000 2 STU_3 70 ... 220.765432 0.600000 3 STU_4 72 ... 230.123456 0.650000 4 STU_5 75 ... 245.887654 0.750000 5 STU_6 78 ... 260.445678 1.000000 [6 rows x 6 columns] {'At Risk': [], 'Average': ['STU_1', 'STU_2', 'STU_3'], 'Good': ['STU_4', 'STU_5', 'STU_6'], 'Top Performer': []} Mean: 71.33 Median: 71.0 Std Dev: 4.5 Correlation: 0.99 Tuple: (np.float64(71.33), np.float64(4.5), np.int64(78)) Final Insight: Stable Academic System	student_id marks ... performance_index normalized_marks 0 STU_1 65 ... 200.112345 0.500000 1 STU_2 68 ... 210.334567 0.550000 2 STU_3 70 ... 220.765432 0.600000 3 STU_4 72 ... 230.123456 0.650000 4 STU_5 75 ... 245.887654 0.750000 5 STU_6 78 ... 260.445678 1.000000 [6 rows x 6 columns] {'At Risk': [], 'Average': ['STU_1', 'STU_2', 'STU_3'], 'Good': ['STU_4', 'STU_5', 'STU_6'], 'Top Performer': []} Mean: 71.33 Median: 71.0 Std Dev: 4.5 Correlation: 0.99 Tuple: (np.float64(71.33), np.float64(4.5), np.int64(78)) Final Insight: Stable Academic System

5. Python Program

Paste your complete Python code below:

```
# Your code here
import random
import math
import numpy as np
import pandas as pd

def create_students(count):
    return [
        (
            f"SID_{i + 1}",
            random.randint(0, 100),
            random.randint(0, 100),
            random.randint(0, 50)
        )
        for i in range(count)
    ]

def categorize(df):
    result = {"At Risk": set(), "Average": set(), "Good": set(), "Top Performer": set()}

    for _, rec in df.iterrows():
        sid, marks, att = rec["student_id"], rec["marks"], rec["attendance"]

        if marks < 40 or att < 50:
            result["At Risk"].add(sid)
        elif 40 <= marks <= 70:
            result["Average"].add(sid)
        elif 71 <= marks <= 90:
            result["Good"].add(sid)
        elif marks > 90 and att > 80:
            result["Top Performer"].add(sid)

    return {k: list(v) for k, v in result.items()}

def compute_analysis(df):
    marks_arr = df["marks"].values

    avg_marks = sum(marks_arr) / len(marks_arr)
    med_marks = np.median(marks_arr)
    std_marks = np.std(marks_arr)

    correlation = np.corrcoef(df["marks"], df["attendance"])[0][1]

    min_val, max_val = np.min(marks_arr), np.max(marks_arr)
    df["norm_marks"] = [(x - min_val) / (max_val - min_val) if max_val != min_val else 0 for x in marks_arr]

    is_consistent = std_marks < 15
```

```

    low_att_count = len([x for x in df["attendance"] if x < 50]) > 3
    top_students = len(df[(df["marks"] > 90) & (df["attendance"] > 80)])
    >= 2

    summary = (avg_marks, std_marks, max_val)

    if is_consistent and not low_att_count and top_students:
        final_status = "Stable Academic System"
    elif low_att_count or not top_students:
        final_status = "Critical Attention Required"
    else:
        final_status = "Moderate Performance"

    return avg_marks, med_marks, std_marks, correlation, summary,
    final_status

n = int(input())

student_records = create_students(n)

df = pd.DataFrame(student_records, columns=["student_id", "marks",
"attendance", "assignment"])

df["performance_index"] = [
    (0.7 * m + 0.3 * a) * math.log(att + 1)
    for m, a, att in zip(df["marks"], df["assignment"], df["attendance"])
]

category_output = categorize(df)

mean_v, median_v, std_v, corr_v, summary, insight = compute_analysis(df)

print(df)
print(category_output)
print("Mean:", mean_v)
print("Median:", median_v)
print("Std Dev:", std_v)
print("Correlation:", corr_v)
print("Tuple:", summary)
print("Final Insight:", insight)

```

6. Learning Outcome

What did you learn from this challenge?

Write here...I learned how to work with structured data using Pandas and perform numerical analysis using NumPy. I understood how to generate and process data using Python modules like random and math. I gained experience in classifying data and applying statistical measures manually and using libraries. I also learned how to analyze patterns and derive meaningful insights from data.

Student Declaration

- I hereby declare that this submission is **my own original work**.

- I have **not used AI tools**, copied code from the internet, or shared code with others.
- I understand that **plagiarism will result in ZERO marks**.

Signed:Bhanu Prakash Reddy Padala

Date: 23/04/2026

Faculty In charge:
--dryasir